DBMS TASK 6

Rules for Reliable and Valid Data

Purpose

This task explains the integrity rules applied to OnlineShoppingDB and provides test statements that demonstrate how MySQL accepts valid records and rejects invalid ones.

A. Constraint Register

Rule	Role in the Database	Example
PRIMARY KEY	Provides a unique identifier for every row.	Products.ProductID
FOREIGN KEY	Requires a referenced record to exist.	Orders.CustomerID references Customers.CustomerID
UNIQUE	Stops duplicate values where uniqueness is required.	Customers.Email
NOT NULL	Requires essential fields to contain a value.	Products.ProductName
CHECK	Restricts values to valid conditions.	Stock >= 0
DEFAULT	Supplies a value when an insert omits the column.	Stock defaults to 0
AUTO_INCREMENT	Generates a new numeric ID automatically.	Orders.OrderID

B. Check the Default Value

```
INSERT INTO Products (ProductName, Category, Price)
VALUES ('Test Mouse', 'Accessories', 500.00);

SELECT ProductName, Stock
FROM Products
WHERE ProductName = 'Test Mouse';
```

Expected: Stock is set to 0 because the INSERT statement leaves that column out.

C. Check Duplicate Email Protection

```
-- This should fail if arun@example.com already exists.
INSERT INTO Customers (CustomerName, Email)
VALUES ('Duplicate Customer', 'arun@example.com');
```

Expected: MySQL reports a UNIQUE constraint violation when the email is already in Customers.

D. Check the Price Rule

```
INSERT INTO Products (ProductName, Category, Price, Stock)
VALUES ('Invalid Product', 'Test', -10.00, 2);
```

Expected: the row is rejected because the product price must not be negative.

E. Check the Customer Reference

```
INSERT INTO Orders (CustomerID, OrderDate, Status)
VALUES (99999, '2026-10-01', 'Pending');
```

Expected: the row is rejected if CustomerID 99999 does not exist.

F. Inspect Table Definitions

```
SHOW CREATE TABLE Customers;
SHOW CREATE TABLE Products;
SHOW CREATE TABLE Orders;
SHOW CREATE TABLE OrderDetails;
SHOW CREATE TABLE Payments;

SELECT o.OrderID, c.CustomerName
FROM Orders AS o
JOIN Customers AS c ON c.CustomerID = o.CustomerID;
```

Run each invalid test separately. Capture both the successful default-value query and the error messages from rejected inserts. Do not include invalid test rows as normal sample data.

Closing Remarks

Constraints help keep records complete, unique, within permitted ranges, and correctly linked. The tests provide evidence that the database rules are enforced.

SQL CODE — MYSQL WORKBENCH

Run tests one at a time. Some inserts are intentionally invalid and should be rejected.

```
USE OnlineShoppingDB;

-- DEFAULT test: Stock should be 0
INSERT INTO Products (ProductName, Category, Price)
VALUES ('Test Mouse', 'Accessories', 500.00);
SELECT ProductName, Stock FROM Products
WHERE ProductName = 'Test Mouse';

-- UNIQUE test: expected to fail if email already exists
INSERT INTO Customers (CustomerName, Email)
VALUES ('Duplicate Customer', 'arun@example.com');

-- CHECK test: expected to fail
INSERT INTO Products (ProductName, Category, Price, Stock)
VALUES ('Invalid Product', 'Electronics', -100.00, 5);

-- FOREIGN KEY test: expected to fail if customer 99999 is absent
INSERT INTO Orders (CustomerID, OrderDate, Status)
VALUES (99999, '2026-10-01', 'Pending');

SHOW CREATE TABLE Customers;
SHOW CREATE TABLE Products;
SHOW CREATE TABLE Orders;
SHOW CREATE TABLE OrderDetails;
SHOW CREATE TABLE Payments;
```